Name: Omar Ahmed Kohla

ID:24P0086

Task 6

Submitted To:

PROF. Gamal Abdel Shafy Ebrahim

ENG. Sally Shaker

Contents

Task 6: Crossing dots (Nine-dot puzzle) 3

Problem 3

- **Definition** 3
- **History of the problem** 3
- **Assumptions** 4
- **Pseudocode and Explanation** 4
- **C++ code** 9
- **Complexity analysis** 12
- **Test cases** 14
- ***Greedy approach*** 16
  - **Algorithm** 16
  - **Pseudocode** 16
  - **Complexity analysis** 16
- **Comparison between the dynamic programming and *greedy*** 18

Conclusion 18

References 19

Task 6: Crossing dots (Nine-dot puzzle)

Problem

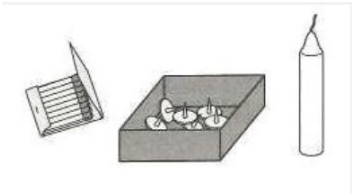
Definition

Given an n×n point lattice (intersection points of n consecutive horizontal and n consecutive vertical lines on common graph paper), where n > 2, cross out all the points by 2n − 2 straight lines without lifting your pen from the paper. You may cross the same point more than once, but you cannot redraw any portion of the same line. (A solution for n =4, shown in the Figure, has seven lines instead of the six required by the puzzle.) **Design a dynamic programming algorithm to solve this problem.**

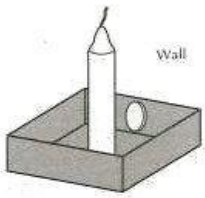
History of the problem

The nine dot problem is a classic lateral thinking exercise that gained widespread popularity in the 1970's and 80's. Participants are presented with a set of dots arranged in a 3x3 grid and challenged to connect all nine dots, without lifting their pencil from the paper, using the fewest possible number of straight lines.

The solution requires one to "think outside the box" and while some contend that the nine dot problem served as the inspiration for this popular turn of phrase, others point to a cognitive performance test from 1945 known as Duncker's candle problem.

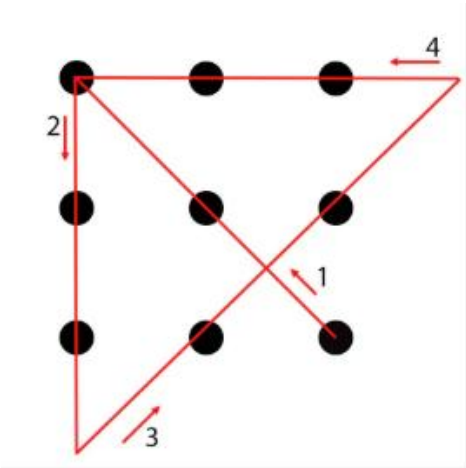


In Duncker's test, participants are presented with a candle, a book of matches and a box full of push pins. The challenge is to affix the candle to the wall in such a way that when the candle is lit, the wax will not drip onto the table... The solution requires the functional use of the box, which at first may seem to be included simply to contain the push pins.



Whether or not the nine dot problem is in fact the original inspiration for the cliched metaphor, the puzzle itself certainly pre-dates the phrase. The first known publication was in Sam Loyd's classic Cyclopedia of Puzzles, 1914.

Though in a 1959 compendium of Sam Loyd's work, Martin Gardner described this particular puzzle as a "classic geometrical challenge", so the nine dots likely predate Loyd's eggs. The enduring aspect of the puzzle is that it highlights the way our minds tend to impose unnecessary limitations upon methods of attacking problems. For those unfamiliar, the most popular solution to the puzzle is depicted below.



Assumptions

- Assume that the grid size is constant, which is 16.

```
const int GRID_SIZE = 16;
```

- The line of the answer is drawn in + for clarification.

Pseudocode and Explanation

The dynamic programming works as follows:

- **Base case** (n=3)
  2*n-2=4 lines
- **Inductive step** (n>3)
  For each solution, we increment by adding 2 additional lines in a structured manner, to make sure it covers all points in the lattice

Assuming first, there is a 16x16 grid to allow the function to draw the answer.

```
ALGORITHM

Function drawcrossing() :

        //initialize an empty board
        // initialize the start position
        SET startPos = center-ish position in big grid
        SET currentPos = startPos
        SET drawPos = startPos
```

The algorithm begins by initializing a large empty grid that will act as the drawing canvas. This grid is intentionally larger than $n \times n$to allow the path to expand without exceeding boundaries.

The variable startPos represents the initial location where the drawing begins.
It is chosen near the centre to allow symmetric movement in all directions.

Two tracking variables are used:

- currentPos: keeps track of the current position of the drawing path.
- drawPos: determines where the $n \times n$ grid will be embedded inside the larger grid.

```
FOR i from 4 to n - 1:
IF i is even:
INCREMENT drawPos.row
ELSE:
DECREMENT drawPos.column
```

This step adjusts the placement of the $n \times n$grid within the larger board.

The adjustment alternates direction:

- For even values of i, The grid shifts downward.
- For odd values, it shifts left.

This ensures that as n grows, the drawing remains centred and does not overflow the grid boundaries.

```
FOR each row i from 0 to n - 1:
  FOR each column j from 0 to n - 1:
    MARK grid cell at calculated position as '.'
```

The algorithm now constructs the visible $n \times n$grid within the larger board.

Each cell is marked with a placeholder (.), indicating that it is part of the grid but has not yet been visited by the drawing path.

BASE CASE: n = 3:

//First line (diagonal upright)

```
REPEAT 3 times:
   MOVE currentPos up
   MOVE currentPos right
   MARK position as visited ('+')
```

The drawing begins by moving diagonally upward to the right.
This establishes the initial direction and covers a sequence of points along a diagonal.

//Second line (downward)

```
REPEAT 3 times:
   MOVE currentPos down
   MARK position as visited('+')
```

The path changes direction and moves vertically downward.
This creates a second segment that connects smoothly with the first.

//Third Line (Diagonal Up-Left)

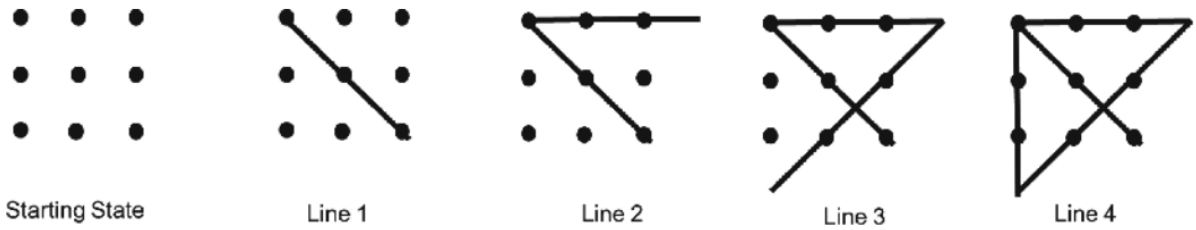
```
REPEAT 3 times:
   MOVE currentPos up
   MOVE currentPos left
   MARK position as visited(‘+’)
```

The path now moves diagonally upward to the left.
This mirrors the first movement and helps form a balanced structure.

//Fourth Line (Rightward)

```
REPEAT 4 times:
   MOVE currentPos right
   MARK position as visited
```

Finally, the path extends horizontally to the right.
This completes the base pattern for $n = 3$, forming a continuous path that covers the required points.



For n>3:

```
SET sizeNow = 4
```

After completing the base case, the algorithm prepares to expand the path for larger grid sizes.
The variable sizeNow represents the current size of the structure being constructed.

```
WHILE sizeNow ≤ n:

  IF right cell of currentPos is not visited:

    REPEAT sizeNow - 1 times:
      MOVE currentPos down
      MARK position

    REPEAT sizeNow times:
      MOVE currentPos left
      MARK position

  ELSE:

    REPEAT sizeNow - 1 times:
      MOVE currentPos up
      MARK position

    REPEAT sizeNow times:
      MOVE currentPos right
      MARK position

  INCREMENT sizeNow
```

This loop extends the path incrementally for increasing grid sizes.

At each iteration:

- The algorithm checks whether the adjacent right cell has been visited.
- Based on this condition, it selects one of two movement patterns:

Case 1:

- Move downward
- Then move left

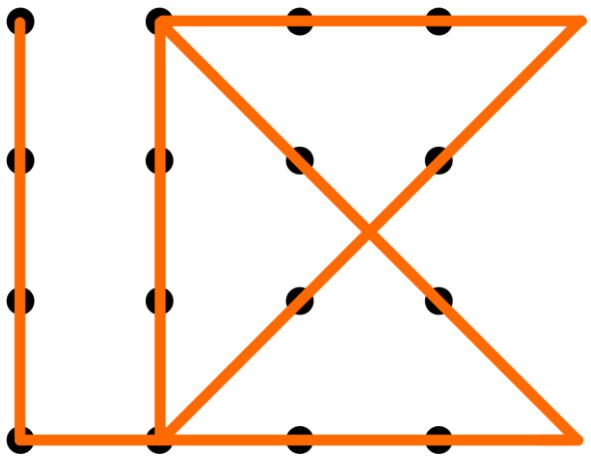
Case 2:

- Move upward
- Then move right

This alternating strategy ensures:

- The path does not overlap previously visited points
- The drawing expands outward in a structured manner
- All required points are eventually covered

The variable sizeNow increases after each iteration, allowing the path to grow layer by layer.



Example for n=4

C++ code

```
#include <iomanip> // for setw() in display grid only
#include <iostream>
using namespace std;
const int GRID_SIZE = 16;
int stepCounter = 1;
// PRINT GRID
void displayGrid(char grid[GRID_SIZE][GRID_SIZE]) {
   for (int r = 0; r < GRID_SIZE; r++) {
      for (int c = 0; c < GRID_SIZE; c++) {
         cout << setw(2) << grid[r][c];
      }
      cout << endl;
   }
   cout << endl;
}
// MAIN ALGORITHM

void drawCrossing(char grid[GRID_SIZE][GRID_SIZE], int originalN) {
   pair<int,int> startPos = {GRID_SIZE / 2 + 2, GRID_SIZE / 2 - 2};
   pair<int,int> endPos;
   pair<int,int> currentPos = startPos;
   pair<int,int> drawPos = startPos;
   //  FIND START CORNER OF GRID
   for (int i = 4; i < originalN; i++) {
      if (i % 2 == 0) drawPos.first++;
      else drawPos.second--;
   }
   //  DRAW BASE GRID
   for (int i = 0; i < originalN; i++) {
      for (int j = 0; j < originalN; j++) {
         grid[drawPos.first - i - 1][drawPos.second + j + 1] = '.';
      }
   }

   displayGrid(grid);
   // BASE CASE (n = 3)

   cout << "Starting n = 3\n\n";

   // Line 1
   for (int i = 0; i < 3; i++) {
      currentPos.first--;
      currentPos.second++;
      grid[currentPos.first][currentPos.second] = '+';
   }
```

```
stepCounter++;
displayGrid(grid);

// Line 2
for (int i = 0; i < 3; i++) {
   currentPos.first++;
   grid[currentPos.first][currentPos.second] = '+';
}
stepCounter++;
displayGrid(grid);
// Line 3
for (int i = 0; i < 3; i++) {
   currentPos.first--;
   currentPos.second--;
   grid[currentPos.first][currentPos.second] = '+';
}
stepCounter++;
displayGrid(grid);
// Line 4
for (int i = 0; i < 4; i++) {
   currentPos.second++;
   grid[currentPos.first][currentPos.second] = '+';
}
stepCounter++;
displayGrid(grid);
out.close();
endPos = currentPos;
cout << "n = 3 finished! Total moves 4\n\n";
if (originalN == 3) return;
int sizeNow = 4;
// INCREMENTAL BUILD

while (sizeNow <= originalN) {

   cout << "Starting n = " << sizeNow << "\n\n";

   // Case 1: go downward then left
   if (grid[currentPos.first][currentPos.second + 1] != '+') {

      for (int i = 0; i < sizeNow - 1; i++) {
         currentPos.first++;
         grid[currentPos.first][currentPos.second] = '+';
      }
      stepCounter++;
      displayGrid(grid);

      for (int i = 0; i < sizeNow; i++) {
```

```
            currentPos.second--;
            grid[currentPos.first][currentPos.second] = '+';
        }
        stepCounter++;
        displayGrid(grid);
    }
    // Case 2: go upward then right
    else if (grid[currentPos.first][currentPos.second - 1] != '+') {

        for (int i = 0; i < sizeNow - 1; i++) {
            currentPos.first--;
            grid[currentPos.first][currentPos.second] = '+';
        }
        stepCounter++;
        displayGrid(grid);

        for (int i = 0; i < sizeNow; i++) {
            currentPos.second++;
            grid[currentPos.first][currentPos.second] = '+';
        }
        stepCounter++;
        displayGrid(grid);
    }
    cout << "n = " << sizeNow << " finished! Total moves " << 2 * sizeNow - 2 << "\n\n";
    sizeNow++;
  }
}
// MAIN
int main() {
  char grid[GRID_SIZE][GRID_SIZE];
  // Initialize grid with spaces
  for (int i = 0; i < GRID_SIZE; i++)
    for (int j = 0; j < GRID_SIZE; j++)
      grid[i][j] = ' ';
  int n;
  cout << "Enter n: ";
  cin >> n;
  drawCrossing(grid, n);
  return 0;
}
```

This program constructs and visualizes a continuous path across an $n \times n$grid using a larger fixed board. The grid is represented as a 2D character array, and a helper function is used to print it in a structured format.

The main algorithm begins by selecting a starting position near the center of the board to allow expansion in all directions. It then adjusts the placement of the $n \times n$grid within the larger board and marks this region using '.' to indicate valid grid points.

For the base case $n = 3$, The algorithm draws a continuous path using four connected segments: diagonal up-right, downward, diagonal up-left, and rightward. Each step updates the current position and marks visited cells with '+', ensuring the path is continuous.

For larger values of n, The algorithm expands the path incrementally. At each step, it checks neighbouring cells to decide the direction of movement. Depending on whether adjacent cells are already visited, it alternates between moving down then left, or up and then right. This strategy avoids overlap and extends the path layer by layer until the entire grid is covered.

Overall, the algorithm builds a structured, continuous path by combining a fixed base pattern with iterative expansion.

Complexity analysis

1. Grid initialization phase:

displaygrid() function which initialize 16x16 grid in $O(1)$ and uses nested loops to display then in $O(n^2)$

Therefore, the time complexity of this function is

$$\boldsymbol{O(n^2)}$$

2. Base case (n=3) phase:

For n=3 phase, the program generates 4 lines:

- diagonal up-right
- downward
- diagonal up-left
- rightward.

Each line is generated in $O(1)$ since they are constant.

Therefore, the time complexity of this function is

$$\boldsymbol{O(1)}$$

3. Inductive step (n>3):

For larger grids the program loops from n=4 to noriginal

- Horizontal line: covers n points, which is $O(\mathrm{n})$
- Vertical line: covers n-1 points, which is $O(\mathrm{n})$

Therefore, this gives us the inductive step recurrence equation:

$$T(n) = T(n-1) + O(n)$$

$$= T(n-2) + O(n-1) + O(n)$$

$$= T(n-3) + O(n-2) + O(n-1) + O(n)$$

$$= T(n-4) + O(n-3) + O(n-2) + O(n-1) + O(n)$$

....

$$T(3) + k = \sum_{k=4}^{n} O(k)$$

$$\sum_{k=4}^{n} k = (1+x)^n = \frac{n(n+1)}{2} - 6$$

Therefore, according to the arithmetic sequence, the time complexity of this phase is:

$$\boldsymbol{O(n^2)}$$

4. Total Complexity

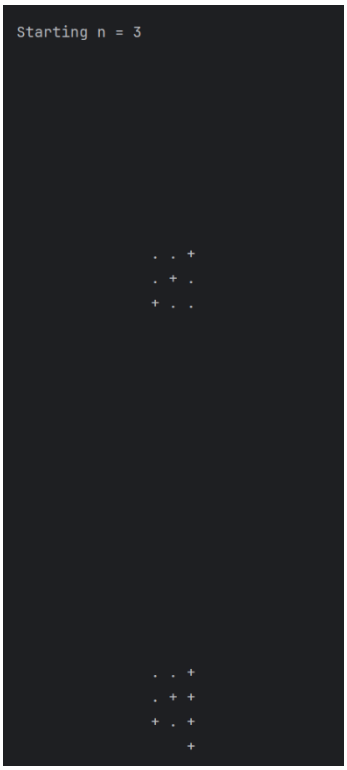
The algorithm has a complexity of quadratic, which is not optimum for bigger n

$$\boldsymbol{O(n^2)}$$

Which comes from $O(n^2)$ in initialization phase, $O(1)$ in the base case and $O(n^2)$ In the inductive step case of calculating the recurrence equation

$T(n) = T(n-1) + O(n)$

<u>Test cases</u>



```
Starting n = 4

  . . . .
+ + + + +
  + + + +
  + + + +
      + +

  . . . .
+ + + + +
  + + + +
  + + + +
+ + + + +
```

```
Starting n = 5

+ . . . . .
+ + + + + .
+ + + + + .
+ + + + + .
+ + + + + .

+ + + + + +
+ + + + + .
+ + + + + .
+ + + + + .
+ + + + + .
```

For a clear and professional visualizer, click here.

Greedy approach

Algorithm

The problem can be approached using a greedy strategy by selecting the next valid unvisited position at each step. A purely greedy approach may not guarantee correctness or optimality, whereas the current algorithm ensures a valid and efficient solution through a predefined movement pattern.

Pseudocode

```
WHILE there are still unvisited grid points:

   IF can move in current direction:
      continue in same direction

   ELSE:
      choose next direction from:
         down, up, left, right, diagonals

      pick first valid unvisited direction

current = start

WHILE path not complete:

   FOR each direction in priority order:
      IF next cell is inside grid AND not visited:
         move there
         mark visited
         BREAK
```

Complexity analysis

1. Number of Steps

Total cells= n^2

So, it is $\boldsymbol{O(n^2)}$

2. Checking Directions at each step

It just checks each direction in constant time.

So, complexity will be $\boldsymbol{O(1)}$

3. Backtracking of the algorithm

This algorithm will backtrack itself, looking for the optimum solution each cycle, so the recurrence relationship of it will be

$$\boldsymbol{T(n) = 1 + 4 + 4^2 + 4^3 + \cdots + 4^{n^2}}$$

$$\boldsymbol{T(n) = \sum_{k=0}^{n^2} 4^k = \frac{\left(4^{n^2+1} - 1\right)}{3}}$$

Therefore, the time complexity of this step will be ≈ $\boldsymbol{O(2^{n^2})}$

3. Total work done by the algorithm

Since we have three phases with complexities of { $\boldsymbol{O(n^2)}$, $\boldsymbol{O(1)}$, $\boldsymbol{O(2^{n^2})}$ **}**

Therefore, the time complexity of this algorithm will be

$$\boldsymbol{O(2^{n^2})}$$

Comparison between the dynamic programming and *greedy*

The key differences between dynamic programming and greedy algorithms are the time complexity and the guarantee of obtaining a correct solution; while dynamic programming ensures an optimal and complete answer by considering all subproblems, a greedy approach makes only local decisions and does not guarantee a valid solution, as it may reach a dead end without exploring alternative paths. From a time complexity perspective, dynamic programming often achieves polynomial time by avoiding redundant computations, whereas greedy algorithms typically run faster with simpler $O(\mathrm{n})$ or $O(n^2)$ time, but may degrade significantly to $O(2^{n^2})$ or fail if the problem does not satisfy the required conditions.

Conclusion

In this research project, we investigated the problem of constructing a continuous path over an (nxn) grid under specific movement constraints, where the objective is to cover grid points systematically without lifting the pen and while maintaining a predictable structure. The study demonstrated how this type of combinatorial grid problem can be modelled and solved using algorithmic design techniques.

The dynamic programming (constructive) approach played a central role in achieving a reliable solution. By building the path incrementally from smaller configurations and extending it step by step, the method ensures consistency, correctness, and scalability as the grid size increases. This structured construction allows the solution to maintain a valid pattern across different values of n, making it suitable for generalisation.

In contrast, alternative strategies such as greedy decision-making were analyzed conceptually, highlighting their limitations in guaranteeing completeness of the path due to their reliance on local choices. Overall, the project shows that structured algorithmic thinking, particularly through dynamic programming principles, provides a robust framework for solving grid traversal and path construction problems in a systematic and mathematically grounded way.

References

1. https://www.geeksforgeeks.org/dsa/dynamic-programming/

2. S. Loyd, Cyclopedia of Puzzles. The Lamb Publishing Company. 1914.

3. https://www.artofplay.com/blogs/stories/history-of-the-nine-dot-problem?srsltid=AfmBOoo8F5uVdvkXhLo-hzHNDNq8_7eGoIzAT3wcxDUYme-AoJFqOThH

4. https://www.researchgate.net/figure/The-nine-dots-problem-and-its-solution-The-task-is-to-cancel-all-the-dots-by-drawing_fig1_385986923

5. Algorithms for Optimization , Mykel J. Kochenderfer and Tim A. Wheeler First Edition, MIT Press, 2019 , section 22.5